# AgentHub Python-First 协作开发文档

> 适用阶段：项目前期开发、三人协作、Codex / Claude Code / TRAE 辅助编程。
> 推荐保存路径：`docs/dev-guide.md`。
> 文档目标：统一项目目标、技术栈、模块边界、协作流程和阶段性验收标准，避免多人使用 AI Coding 时出现代码风格不一致、模块耦合混乱和无法集成的问题。

---

## 1. 项目定位

AgentHub 是一个面向 AI 开发协作场景的 IM 聊天式多 Agent 协作平台。用户可以像使用飞书或微信一样创建会话、发送消息、通过 `@Agent` 指令指定不同 AI Agent 参与任务，并由 Orchestrator 对复杂任务进行拆解、分派、执行和汇总。

本项目采用 Python-first 开发路线：

- 后端、Agent 调度、Agent Adapter、Workspace、Diff、预览服务全部优先使用 Python 实现；
- 前端可根据团队能力选择 Python Web UI 或 React Web UI；
- 比赛 Demo 目标优先于工程大而全；
- 所有 AI Coding 过程必须有 Prompt 记录、人工 Review 和 PR 流程。

项目最终目标是交付一个可运行 Demo，并能在答辩中清楚解释产品设计、技术架构、AI 协作方法和核心代码逻辑。

---

## 2. 技术路线

### 2.1 推荐方案

本项目建议采用：

```
Python 后端 / Agent 核心 + Web 前端
```

后端和 Agent 核心使用 Python，前端可以有两种选择：

#### 方案 A：Python 全栈 Demo 方案

适合团队希望尽量统一 Python 技术栈。

推荐组合：

- FastAPI：后端 API；
- NiceGUI / Reflex：快速构建 Web 界面；
- SQLite：本地数据库；
- SQLModel / SQLAlchemy：ORM；
- Pydantic v2：数据模型；
- pytest：测试；
- ruff：格式化与 lint；
- mypy / pyright：类型检查。

优点：

- 技术栈统一；
- 三人都围绕 Python 协作；
- Demo 开发速度快；
- 后端与 Agent 逻辑衔接自然。

缺点：

- IM 级交互和前端精细度可能弱于 React；
- 复杂代码编辑器、Diff 视图和 iframe 预览组件需要额外处理。

### 方案 B：React 前端 + Python 后端方案（推荐）

适合更重视比赛展示效果和产品体验。

推荐组合：

- 前端：React / Next.js / Vite + Tailwind；
- 后端：FastAPI；
- Agent 核心：Python；
- 数据库：SQLite + SQLModel；
- 流式消息：SSE；
- Workspace：Python 文件系统管理；
- Diff：Python 生成 unified diff，前端展示；
- 预览：后端启动静态服务或 Vite 服务，前端 iframe 展示。

优点：

- UI 效果更好；

- 更适合展示 IM 聊天、卡片、Diff、预览面板；
- Python 仍然承担项目核心逻辑。

缺点：

- 多一个前端技术栈；
- 需要定义清晰的前后端接口。

### 2.2 本文档默认选择

为了兼顾比赛展示效果和 Python 核心开发，本文档默认采用：

`React 前端 + FastAPI 后端 + Python Agent 核心`

如果团队坚持全 Python，可以将前端部分替换为 NiceGUI / Reflex，但后端、Agent、Workspace、数据模型和测试规范仍按本文档执行。

---

## 3. MVP 范围

### 3.1 P0：必须完成

P0 是项目最小可演示闭环，必须优先完成。

| 模块 | 功能 |
| --- | --- |
| 会话系统 | 新建会话、会话列表、切换会话、保存消息历史 |
| 聊天系统 | 用户消息、Agent 消息、系统消息、代码块展示 |
| Agent 系统 | 至少包含 Orchestrator、Code Agent、Review Agent |
| @ 指令 | 识别 `@AgentName` 并路由到指定 Agent |
| Orchestrator | 对复杂任务进行拆解、分派和汇总 |
| Adapter 层 | 所有 Agent 通过统一 Python 接口调用 |
| Artifact 卡片 | 支持代码文件、Diff、网页预览三类产物 |
| Workspace | 每个会话绑定一个本地工作区，可读写文件 |
| Demo 主线 | 用户提出网页开发需求，系统完成拆解、代码生成、Diff、应用和预览 |

### 3.2 P1：建议完成

| 模块 | 功能 |
| --- | --- |
| SSE 流式输出 | Agent 回复逐步显示 |
| Apply Diff | 用户点击按钮后应用代码变更 |
| Git Commit | 每次应用变更后自动生成 commit |
| 真实 Agent 接入 | 至少接入一个真实模型 / Coding Agent |
| Mock Fallback | 真实 Agent 失败时回退到 Mock Agent |
| Prompt Log | 记录关键 AI Coding 过程 |

### 3.3 P2：后续扩展

以下功能可写入设计文档，但不作为前期开发重点：

1. 真正的一键云部署；
2. 多人实时协作；
3. 移动端和桌面端；
4. 用户权限系统；
5. 完整版本历史；
6. 用户自定义 Agent 创建器；
7. 复杂并行调度和冲突自动解决。

---

## 4. 推荐目录结构

```
AgentHub/
├── agenthub/
│   ├── __init__.py
│   ├── main.py                    # FastAPI 入口
│   ├── config.py                  # 配置管理
│   ├── api/                       # API 路由层
│   │   ├── __init__.py
│   │   ├── conversations.py
│   │   ├── messages.py
│   │   ├── agents.py
│   │   └── workspaces.py
│   ├── schemas/                   # Pydantic 入参/出参模型
│   │   ├── __init__.py
│   │   ├── conversation.py
│   │   ├── message.py
│   │   ├── agent.py
│   │   └── artifact.py
│   ├── models/                    # 数据库模型
```

```
│   │   ├── __init__.py
│   │   ├── conversation.py
│   │   ├── message.py
│   │   └── workspace.py
│   ├── services/                       # 业务服务层
│   │   ├── __init__.py
│   │   ├── conversation_service.py
│   │   ├── message_service.py
│   │   └── agent_service.py
│   ├── agents/                         # Agent Adapter 层
│   │   ├── __init__.py
│   │   ├── base.py
│   │   ├── mock_agent.py
│   │   ├── code_agent.py
│   │   ├── review_agent.py
│   │   └── real_agent.py
│   ├── orchestrator/                   # 调度器
│   │   ├── __init__.py
│   │   ├── planner.py
│   │   ├── router.py
│   │   └── executor.py
│   ├── workspace/                      # 工作区、文件、Diff、Git、预览
│   │   ├── __init__.py
│   │   ├── manager.py
│   │   ├── diff.py
│   │   ├── git.py
│   │   └── preview.py
│   └── utils/
│       ├── __init__.py
│       ├── ids.py
│       └── security.py
├── frontend/                           # 可选：React / Next.js / Vite 前
端
├── workspaces/                         # 本地生成的项目工作区，通常不提交
├── tests/
│   ├── test_agents.py
│   ├── test_orchestrator.py
│   ├── test_workspace.py
│   └── test_api.py
├── docs/
│   ├── dev-guide.md
│   ├── product-spec.md
│   ├── tech-design.md
│   ├── ai-rules.md
│   ├── prompt-log.md
│   └── demo-script.md
├── mock/
│   └── demo_responses/
├── .github/
```

```
│   └── pull_request_template.md
├── AGENTS.md
├── README.md
├── pyproject.toml
├── .env.example
└── .gitignore
```

---

## 5. 核心数据模型

所有 API 入参和出参必须使用 Pydantic 模型，不允许直接返回未经定义的 dict。

### 5.1 Conversation

```python
from pydantic import BaseModel
from typing import Literal

class Conversation(BaseModel):
    id: str
    title: str
    mode: Literal["single", "group"]
    agent_ids: list[str]
    workspace_id: str
    created_at: str
    updated_at: str
```

### 5.2 Message

```python
from pydantic import BaseModel
from typing import Literal

class Message(BaseModel):
    id: str
    conversation_id: str
    role: Literal["user", "agent", "system"]
    sender_id: str
    content: str
    artifacts: list["Artifact"] = []
    created_at: str
```

### 5.3 Agent

```python
from pydantic import BaseModel
from typing import Literal

class Agent(BaseModel):
    id: str
    name: str
    role: Literal["orchestrator", "code", "review", "preview", "custom"]
    description: str
```

```
    capabilities: list[str]
    adapter_type: Literal["mock", "codex", "claude", "opencode", "custo
m"]
```

### 5.4 Artifact

```python
from pydantic import BaseModel
from typing import Any, Literal

class Artifact(BaseModel):
    id: str
    type: Literal["code", "diff", "preview", "file", "deploy"]
    title: str
    data: dict[str, Any]
    created_at: str
```

---

## 6. Agent Adapter 设计

所有 Agent 必须通过统一接口调用，业务层不允许直接依赖具体模型平台。

### 6.1 基础接口

```python
from typing import Protocol, Any
from pydantic import BaseModel

class AgentRunInput(BaseModel):
    conversation_id: str
    workspace_id: str
    user_message: str
    history: list[dict[str, Any]]
    context: dict[str, Any] = {}

class AgentRunResult(BaseModel):
    agent_id: str
    content: str
    artifacts: list[dict[str, Any]] = []
    next_actions: list[str] = []

class AgentAdapter(Protocol):
    id: str
    name: str
    capabilities: list[str]

    async def run(self, input_data: AgentRunInput) -> AgentRunResult:
        ...
```

### 6.2 Mock Agent

Mock Agent 用于保证 Demo 稳定，不应被视为临时代码，而是正式的 fallback 机制。

适用场景：

1. 没有真实 API Key；
2. 真实 Agent 调用失败；
3. Demo 录制；
4. 离线开发；
5. 前后端联调。

### 6.3 Real Agent

真实 Agent 接入要求：

1. API Key 只能放在 `.env` 中；
2. 所有调用必须经过后端；
3. 必须设置 timeout；
4. 必须捕获异常；
5. 失败时返回可读错误；
6. 支持 fallback 到 Mock Agent。

示例规则：

```
try:
    result = await real_agent.run(input_data)
except Exception:
    result = await mock_agent.run(input_data)
```

---

## 7. Orchestrator 设计

Orchestrator 是群聊模式下的主 Agent，负责理解用户意图、拆解任务、选择子 Agent、执行任务并汇总产出。

### 7.1 基础流程

```
用户输入需求
↓
读取会话历史和 workspace 状态
↓
Orchestrator 生成任务计划
↓
```

```
选择需要调用的子 Agent
↓
调用 Code Agent / Review Agent / Preview Agent
↓
收集结果和 Artifact
↓
汇总为聊天消息
```

## 7.2 第一版调度规则

前期不做复杂智能调度，优先使用规则驱动。

```
如果消息包含“写 / 实现 / 生成 / 页面 / 组件”
→ 调用 Code Agent

如果消息包含“检查 / 优化 / bug / review / 问题”
→ 调用 Review Agent

如果消息包含“预览 / 效果 / 页面展示”
→ 调用 Preview Agent

如果用户明确 @ 某个 Agent
→ 优先调用指定 Agent

如果用户 @Orchestrator
→ Orchestrator 拆解任务并调度多个 Agent
```

## 7.3 任务计划模型

```python
from pydantic import BaseModel

class AgentTask(BaseModel):
    task_id: str
    agent_id: str
    instruction: str
    depends_on: list[str] = []

class OrchestratorPlan(BaseModel):
    summary: str
    tasks: list[AgentTask]
```

---

# 8. Workspace、Diff 与 Git

## 8.1 Workspace 设计目标

每个会话绑定一个独立 workspace，用于保存该会话中 Agent 生成的代码文件、配置文件和预览文件。

```
workspaces/
├── conv_001/
│   ├── src/
│   ├── package.json
│   └── index.html
└── conv_002/
```

## 8.2 文件安全规则

所有文件读写必须遵守：

1. 使用 `pathlib.Path`；
2. 禁止路径穿越；
3. 禁止访问 workspace 目录之外的文件；
4. 禁止使用用户输入直接拼接系统命令；
5. 禁止在 subprocess 中使用 `shell=True`。

## 8.3 Diff 生成

使用 Python 标准库 `difflib` 生成 unified diff。

```python
import difflib

def create_unified_diff(old: str, new: str, filename: str) -> str:
    return "".join(
        difflib.unified_diff(
            old.splitlines(keepends=True),
            new.splitlines(keepends=True),
            fromfile=f"a/{filename}",
            tofile=f"b/{filename}",
        )
    )
```

## 8.4 Git 操作

Git 操作必须封装在 `workspace/git.py` 中，不允许散落在业务代码中。

允许操作：

```
git init
git status
```

```
git diff
git add
git commit
git log
```

要求：

1. 所有命令必须使用 list 参数；
2. 必须设置 cwd；
3. 必须捕获错误；
4. commit message 必须清楚。

---

## 9. API 设计

### 9.1 会话 API

```
GET    /api/conversations
POST   /api/conversations
GET    /api/conversations/{conversation_id}
PATCH  /api/conversations/{conversation_id}
DELETE /api/conversations/{conversation_id}
```

### 9.2 消息 API

```
GET  /api/conversations/{conversation_id}/messages
POST /api/conversations/{conversation_id}/messages
```

### 9.3 Agent API

```
POST /api/agents/run
POST /api/agents/stream
```

请求示例：

```
{
  "conversation_id": "conv_001",
  "agent_id": "orchestrator",
  "message": "@Orchestrator 帮我做一个 Todo List 网页"
}
```

### 9.4 Workspace API

```
GET  /api/workspaces/{workspace_id}/files
GET  /api/workspaces/{workspace_id}/files/content?path=src/App.py
POST /api/workspaces/{workspace_id}/files
POST /api/workspaces/{workspace_id}/apply-diff
GET  /api/workspaces/{workspace_id}/preview
POST /api/workspaces/{workspace_id}/git/commit
```

## 10. 三人分工

### 成员 A：前端与产品体验

负责：

1. IM 主界面；
2. 会话列表；
3. 消息流；
4. @Agent 输入体验；
5. Agent 头像和能力标签；
6. Artifact 卡片；
7. 预览面板；
8. Demo 视觉优化。

主要目录：

```
frontend/
docs/product-spec.md
docs/demo-script.md
```

### 成员 B：FastAPI 后端与 Agent 系统

负责：

1. FastAPI 应用入口；
2. 会话 API；
3. 消息 API；
4. Agent Adapter；
5. Mock Agent；
6. Real Agent 接入；
7. SSE 流式输出。

主要目录：

```
agenthub/api/
agenthub/services/
agenthub/agents/
agenthub/schemas/
```

### 成员 C：Orchestrator、Workspace、Diff 与 Git

负责：

1. Orchestrator 任务拆解；
2. Agent 路由规则；
3. workspace 文件管理；
4. Diff 生成和应用；
5. Git commit；
6. 本地预览服务。

主要目录：

```
agenthub/orchestrator/
agenthub/workspace/
tests/test_orchestrator.py
tests/test_workspace.py
```

---

## 11. GitHub 协作规范

### 11.1 分支规则

```
main                  # 稳定可演示版本
dev                   # 日常集成分支
feature/api           # FastAPI 接口
feature/agents        # Agent Adapter
feature/orchestrator  # 调度器
feature/workspace     # Workspace / Diff / Git
feature/frontend      # 前端界面
docs/spec             # 文档
```

### 11.2 开发流程

```
git checkout dev
git pull origin dev
git checkout -b feature/xxx
```

开发完成：

```
git add .
git commit -m "feat: add xxx"
git push origin feature/xxx
```

然后在 GitHub 创建 Pull Request，目标分支为 dev。

### 11.3 合并要求

1. 不允许直接 push 到 `main`；
2. 每个 PR 只解决一个明确问题；
3. PR 必须说明修改内容、测试方式和风险；
4. 至少一名队友 review 后再合并；
5. 合并前必须能本地启动；
6. 每天至少同步一次 `dev`；
7. 每个稳定节点从 `dev` 合并到 `main`。

### 11.4 Commit 规范

```
feat: 新功能
fix: 修复 bug
docs: 文档修改
style: 格式调整
refactor: 重构
test: 测试相关
chore: 工程配置
```

---

## 12. Python 代码规范

### 12.1 基础要求

1. 所有函数必须尽量写类型注解；
2. API 入参和出参必须使用 Pydantic；
3. FastAPI route 只做参数接收和 service 调用，不写复杂业务逻辑；
4. 业务逻辑放在 `services/`、`agents/`、`orchestrator/`、`workspace/`；
5. 文件路径使用 `pathlib.Path`；
6. 外部 HTTP 调用使用 `httpx` 并设置 timeout；
7. 不允许硬编码 API Key；
8. 不允许提交 `.env`；
9. 不允许提交 `workspaces/` 中的临时文件；
10. 不允许使用无意义的变量名和大段复制粘贴代码。

### 12.2 推荐工具

```
ruff check .
ruff format .
pytest
mypy agenthub
```

### 12.3 错误处理

所有关键模块必须处理异常：

1. Agent 调用失败；
2. API Key 缺失；
3. workspace 文件不存在；
4. Diff 应用失败；
5. Git 命令失败；
6. 预览服务启动失败。

---

## 13. Codex / AI Coding 规范

### 13.1 每次使用 Codex 前必须说明

1. 当前任务；
2. 允许修改的文件；
3. 禁止修改的文件；
4. 输入输出接口；
5. 验收标准；
6. 是否需要测试；
7. 是否需要更新文档。

### 13.2 标准 Prompt 模板

```
你是 AgentHub 项目的 Python 开发助手。

项目目标：
构建一个 IM 式多 Agent 协作平台，核心能力包括会话、@Agent、Orchestrator 调度、Agent Adapter、Workspace、Diff 和网页预览。

当前任务：
实现【填写具体功能】。

技术约束：
1. 后端使用 FastAPI；
2. 数据模型使用 Pydantic v2；
3. 业务逻辑不要写在 route 中；
4. 文件路径必须使用 pathlib.Path；
5. subprocess 禁止 shell=True；
6. 外部请求必须设置 timeout；
7. 不要硬编码 API Key；
```

8. 代码必须尽量包含类型注解。

允许修改文件:
- xxx

禁止修改文件:
- xxx

验收标准:
1. xxx;
2. xxx;
3. 通过 `pytest / ruff` 检查。

完成后请说明:
1. 修改了哪些文件;
2. 核心实现逻辑;
3. 如何测试;
4. 潜在风险。

### 13.3 AI 输出后必须检查

1. 是否修改了不该修改的文件;
2. 是否破坏已有接口;
3. 是否有伪代码;
4. 是否缺少类型注解;
5. 是否缺少异常处理;
6. 是否存在路径越权风险;
7. 是否出现硬编码密钥;
8. 是否需要补充测试;
9. 是否需要更新文档。

---

## 14. 开发里程碑

### 阶段 1：项目初始化

目标:

1. 创建 GitHub 仓库;
2. 初始化 Python 项目;
3. 配置 `pyproject.toml`;
4. 初始化 FastAPI;

5. 创建基础目录；
6. 建立 AGENTS.md 和协作文档。

验收：

1. `uvicorn agenthub.main:app --reload` 可启动；
2. `/health` 接口返回正常；
3. `pytest` 可运行；
4. `ruff check .` 可运行；
5. 三人完成第一次 PR。

## 阶段 2：会话与消息

目标：

1. 实现会话 API；
2. 实现消息 API；
3. 支持创建会话和发送消息；
4. 前端可以展示消息流。

验收：

1. 用户可以创建会话；
2. 用户可以发送消息；
3. 后端能保存消息；
4. 前端能显示用户消息和 Agent 消息。

## 阶段 3：Agent Adapter 与 Mock Agent

目标：

1. 定义 AgentAdapter 接口；
2. 实现 MockAgent；
3. 实现 CodeAgent、ReviewAgent、PreviewAgent 的基础版本；
4. 支持 `@Agent` 路由。

验收：

1. 用户 `@CodeAgent` 可以得到代码回复；
2. 用户 `@ReviewAgent` 可以得到检查建议；
3. Mock Agent 输出稳定；
4. API 返回结构统一。

## 阶段 4：Orchestrator 多 Agent 协作

目标：

1. 实现 Orchestrator 任务拆解；
2. 实现 Agent 路由；
3. 多个 Agent 依次执行；
4. Orchestrator 汇总结果。

验收：

1. 用户 `@Orchestrator` 提出复杂任务；
2. 系统输出任务拆解；
3. Code Agent / Review Agent / Preview Agent 依次返回；
4. 聊天流保留完整过程。

## 阶段 5：Workspace、Diff 与预览

目标：

1. 每个会话创建 workspace；
2. Agent 可以生成文件；
3. 系统可以生成 Diff；
4. 用户可以 Apply Diff；
5. 页面可以预览。

验收：

1. 文件可以写入 workspace；
2. Diff 卡片可以展示；
3. Apply Diff 后文件内容更新；
4. 预览页面可以打开。

## 阶段 6：包装与答辩

目标：

1. 固化 Demo 主线；
2. 完成产品文档；
3. 完成技术文档；
4. 完成 AI 协作记录；
5. 录制 3 分钟 Demo 视频；

6. 准备答辩 Q&A。

验收：

1. `main` 分支可稳定运行；
2. README 包含启动方式；
3. Demo 视频展示完整闭环；
4. 每个成员能解释自己模块。

---

## 15. Demo 主线

推荐演示流程：

1. 用户新建会话：`Todo App 开发`
2. 用户输入：`@Orchestrator 帮我做一个简洁的 Todo List 网页`
3. `Orchestrator 输出任务拆解`
4. `Code Agent 生成代码文件`
5. 系统展示 `Diff 卡片`
6. 用户点击 `Apply Diff`
7. `Review Agent 检查代码并给出建议`
8. `Preview Agent 返回网页预览卡片`
9. 用户继续输入：把按钮改得更醒目
10. 系统生成新的 `Diff 并更新预览`

这条主线必须优先保证稳定，其他功能都不能影响它。

---

## 16. 验收清单

### 16.1 功能验收

- ☐ 可以创建和切换会话
- ☐ 可以发送用户消息
- ☐ 可以展示 Agent 回复
- ☐ 支持 `@Agent`
- ☐ Orchestrator 可以拆解任务
- ☐ 至少两个子 Agent 可以被调用
- ☐ 可以展示代码文件 Artifact
- ☐ 可以展示 Diff Artifact
- ☐ 可以展示 Preview Artifact
- ☐ 可以保存会话历史

- ☐ Demo 主线可以连续跑通

### 16.2 工程验收

- ☐ FastAPI 可以启动
- ☐ pytest 可以运行
- ☐ ruff 可以运行
- ☐ `.env.example` 已提供
- ☐ README 包含启动方式
- ☐ 不暴露 API Key
- ☐ workspace 有路径安全校验
- ☐ PR 流程规范
- ☐ 关键模块有文档说明

### 16.3 交付物验收

- ☐ 产品设计文档
- ☐ 技术设计文档
- ☐ 可运行 Demo
- ☐ AI 协作开发记录
- ☐ 3 分钟 Demo 视频
- ☐ GitHub 仓库
- ☐ 答辩 PPT 或讲稿

---

## 17. 风险与应对

| 风险 | 应对 |
| --- | --- |
| 真实 Agent 接入不稳定 | 保留 Mock Agent，Demo 优先使用稳定链路 |
| Python 前端表现不足 | 必要时使用 React 前端，Python 保留核心逻辑 |
| 三人代码冲突 | 按模块分工，所有功能走 PR |
| AI 生成代码不可控 | 使用 AGENTS.md、任务模板和 Review 清单 |
| 文件操作有安全风险 | workspace 路径校验，禁止访问目录外文件 |
| Demo 临场失败 | 准备 Mock 数据和录制视频 |
| 时间不足 | 只保 P0 闭环，P2 写入设计规划 |

---

## 18. 当前优先任务

第一周优先完成：

1. 创建 GitHub 仓库；
2. 初始化 Python 项目；
3. 配置 FastAPI、pytest、ruff；
4. 创建 `agenthub/` 基础目录；
5. 创建 `AGENTS.md` 和协作文档；
6. 实现 `/health` 接口；
7. 定义 Pydantic 数据模型；
8. 实现 MockAgent；
9. 确认前端方案；
10. 每位成员完成一次独立 PR。

---

## 19. 最小成功标准

最终 Demo 只要稳定完成下面这条链路，就视为项目核心成功：

用户提出开发需求
→ `Orchestrator` 拆解任务
→ 多个 `Agent` 依次产出
→ 系统展示代码和 `Diff`
→ 用户应用变更
→ 页面预览更新
→ 聊天记录保留完整协作过程

后续所有开发都必须围绕这条链路展开，避免为了炫技偏离主线。